Streaming programming — lecciones aprendidas y guion de entrevista

Notas de estudio personales, no documentación formal del proyecto. Sintetiza qué aprendimos de verdad sobre streaming programming a lo largo de las Fases 1-5, dónde hay huecos reales que no conviene maquillar, y cómo responder si un CTO o entrevistador senior pregunta por ello. Todo está enlazado a la fuente original (ADR, spec o incidente) — si algo suena a interpretación, es que lo es y está marcado como tal.

> *Distinto del `lessons-learned` formal que la Fase 7 tiene pendiente escribir (`docs/phase-7-demo-docs-design-decisions.md` , Decisión I) — aquel cubrirá todo el proyecto (Iceberg, dbt, LocalStack, proceso); este documento es solo la parte de streaming, para estudiar antes de una entrevista.*

---

1. Lo que realmente aprendimos (con evidencia, no intuición)

El meta-patrón que se repite tres veces — el más vendible en entrevista

Un health-check en verde no significa que el dato fluya. Pasó tres veces, con formas distintas:

- El connector de Debezium reportó `RUNNING` publicando **cero mensajes** — el topic real y el de la spec habían divergido en silencio. `error-handling/debezium-default-topic-naming-mismatch.md`
- El mismo `RUNNING` con un topic de heartbeat inexistente bloqueando **todo el producer**, no solo los heartbeats. `error-handling/debezium-heartbeat-topic-stalls-entire-connector.md`
- `describe_table` de DynamoDB devolviendo un `LatestStreamArn` que ya **no existía de verdad**. `error-handling/dynamodb-streams-orphaned-after-localstack-restart-needs-full-table-recreation.md`

Lección operativa: todo control-plane check necesita un data-plane check al lado — contar mensajes reales, no confiar en el estado que el propio sistema informa de sí mismo.

CDC / Debezium

- **Identidad estable (`event_id`) + upsert, nunca sumar mensajes.** ADR-0003 — colapsa deduplicación y detección de update en un solo problema; un replay de checkpoint de Flink es inofensivo por construcción.
- **Los valores por defecto de un conector CDC nunca son neutrales.** La clave de partición por defecto de Debezium es la PK de la tabla, no la columna de negocio — rompió la garantía de afinidad de ADR-0001 durante semanas sin que nadie lo notara, hasta que un AC lo verificó **estructuralmente** (agrupar y contar), no por muestreo anecdótico. `error-handling/debezium-default-key-breaks-partition-affinity.md`
- **El offset/snapshot de Kafka Connect tiene memoria que el config actual no refleja.** Añadir una tabla a un connector ya vivo no la snapshotea retroactivamente (`debezium-adding-a-table-to-a-running-connector-skips-its-snapshot.md`), y arreglar la codificación de un connector **no corrige mensajes ya comprometidos** con la codificación vieja (postscript de `debezium-date-decimal-wire-encoding-mismatch.md`).

Doble bus de eventos (Kafka + Kinesis)

- **Una ADR documenta una premisa, no la garantiza para siempre — y por eso vale la pena escribirla.** ADR-0001 asumió que ambos conectores existían en PyFlink 2.x. Era falso: Kinesis no

tiene conector sobre la nueva Source API en Flink 2.x (`flink-2x-removes-legacy-sourcefunction-breaking-flinkkinesisconsumer.md`). Se resolvió con un bridge dedicado (ADR-0008), no bajando de versión.
- **La retención (24h en Kinesis/DynamoDB Streams, la de Kafka también) no es un detalle — es data permanentemente irrecuperable** si un consumidor arranca tarde o cae más tiempo del que dura la ventana. `error-handling/kafka-retention-already-expired-old-payment-lines-history.md` , ADR-0006. No es un bug, es una propiedad estructural del streaming que hay que monitorizar, no "arreglar".

Flink — estado, tiempo, timers (el núcleo del proyecto)

- El join coste⋈mercado no era 1:1, era **fan-out por producto cruzado dentro de un segmento** — resolverlo a mano (`KeyedProcessFunction` , dos `MapState`) en vez de Table API fue la decisión correcta para entender qué guarda Flink y cuándo emite, no solo para que funcionara. Decisión D, `docs/phase-4-streaming-design-decisions.md` .
- El **Broadcast State Pattern garantiza orden dentro del lado broadcast, no entre broadcast y keyed** — una race de arranque real y aceptada explícitamente, no un bug a perseguir. `error-handling/flink-operational-checklist.md` §2.
- `max parallelism` no fijado explícitamente se congela en el primer checkpoint y limita todo rescale futuro — casi se repite un tercer default implícito que muerde después, y esta vez **se implementó la mitigación antes de que ocurriera** (`env.set_max_parallelism` , `streaming/flink-jobs/src/flink_jobs/job.py`).
- Los riesgos previstos por escrito **antes de implementar** se convirtieron en código real, con test:
  - "Stale overwrite" (Riesgo 2 de `anticipated-risks-flink-processing.md`) → `streaming/flink-jobs/src/flink_jobs/staleness.py` + `tests/test_staleness.py` .
  - Eviction cap del `MapState` (Riesgo 3, mismo documento) → `streaming/flink-jobs/src/flink_jobs/eviction.py` + `tests/test_eviction.py` .
  - Esto es la prueba más fuerte de que el pre-spec no fue teatro: el riesgo anticipado se volvió código verificable, no una nota que se quedó en el documento.
- **Y se verificó con caos real, no solo con diseño:**
  - `docker kill` al TaskManager en pleno procesamiento → `"latest restored"` apuntando a un checkpoint S3 real, `cost_lines_count` de un apartamento siguió creciendo tras el fallo, sin resetearse ni duplicarse (AC-08, `docs/AUDIT_DIARY.md`).
  - `docker kill` al consumer de Iceberg a mitad de stream, un registro más escrito mientras estaba caído, reinicio → resume desde el checkpoint guardado en DynamoDB, exactamente las filas esperadas, sin duplicar ni perder ninguna (AC-04, Fase 5, mismo diario).

Dual-write y CDC como patrón, no como accidente

ADR-0006 rechaza explícitamente que el mismo job de Flink escriba a DynamoDB e Iceberg — es el problema de dual-write de Kleppmann (*DDIA*, cap. 11): sin atomicidad entre sistemas, sin forma de detectar divergencia tras un fallo parcial. La solución (un único writer + derivar el segundo sistema vía su propio change log) es el mismo patrón que Debezium ya usa desde la Fase 2 (leer el WAL de Postgres en vez de que la app escriba a los dos), aplicado dos veces en el mismo proyecto — arquitectura consistente, no coincidencia.

LocalStack como fuente de "falsos incidentes de streaming"

Dos incidentes distintos de stream de DynamoDB huérfano, y **el fix documentado del primero no resolvió el segundo** — hubo que escalar a resetear el volumen completo en vez de iterar variaciones del mismo

comando. `error-handling/dynamodb-streams-delete-recreate-cycle-doesnt-fix-orphaned-stream.md`

Lección meta: un fix documentado es una pista, no una garantía — si no resuelve el síntoma la segunda vez, la corrupción vive un nivel más arriba de donde se aplicó el fix.

---

2. Qué se debería reforzar (honestamente, sin maquillar)

Lo que un CTO senior va a preguntar y donde hay que responder con la verdad, no con relleno:

1. **Event-time y watermarks nunca se tocaron de verdad.** Se evitaron deliberadamente (Decisión A del pre-spec de Fase 4) porque el problema no tenía ventanas que cerrar — correcto para *este* job, pero significa cero práctica real con out-of-order handling, `WatermarkStrategy` o windowed aggregation. Del trío "State, Time, Windows", solo State y Timers se cubrieron de verdad.
2. **Cero observabilidad real.** Cada incidente se diagnosticó a mano (`curl` a un status endpoint, logs de Kafka Connect, `describe-table`). Prometheus/Grafana se descartó explícitamente como "sobre-ingeniería para el PoC" — defendible para un PoC, pero es el hueco más grande frente a "esto en producción".
3. **Nunca se probó bajo carga real.** 100 apartamentos, 18 segmentos, volumen sintético pequeño. El hot-shard de ADR-0005 es un riesgo *aceptado y documentado*, no *observado* bajo tráfico real — no hay prueba de backpressure.
4. **Un bug real, encontrado en revisión, sigue sin arreglar a propósito.** El fan-out del lado de coste no filtra noches ya pasadas como sí lo hace el lado de mercado — no muerde en la práctica porque el ingestor de mercado barre las 18 combinaciones constantemente, pero es una condición de carrera latente, diferida a propósito, no descubierta ahora. `error-handling/stage-b-cost-side-fanout-can-emit-for-already-past-nights.md`
5. **Schema evolution nunca se vivió de verdad.** Los contratos tienen `schema_version` , pero jamás hubo un v1→v2 real con productores y consumidores viejos y nuevos coexistiendo. Músculo sin ejercitar.
6. **Nada de esto se verificó contra AWS real todavía** (Fase 7, pendiente). Varios incidentes son específicos del emulador (LocalStack), no del servicio real — Glue Ultimate-tier-only en Community, streams huérfanos tras un restart de contenedor. Parte de la "dureza" observada es del emulador, no necesariamente representativa de AWS real.

---

3. Guion para la entrevista

Respuesta corta (~90 segundos, para hablar)

> *La lección más importante no fue de una tecnología concreta, sino un patrón que se repitió tres veces con formas distintas: un health-check en verde no dice nada sobre si el dato fluye donde debería. Tuvimos un connector de Debezium en estado `RUNNING` publicando cero mensajes porque el topic real y el de la spec habían divergido en silencio, y el mismo estado `RUNNING` con un topic de heartbeat inexistente bloqueando todo el producer. La respuesta no fue "confiar más", fue añadir siempre una verificación en el plano de datos — contar mensajes reales — al lado de cualquier chequeo de control-plane.*
>
> *A nivel de Flink, lo más defendible es que tratamos el estado como el problema real: identidad estable para upserts idempotentes, un modelo explícito de "dos hojas" para el fan-out del join, límites de tamaño en el `MapState` y en el `max parallelism` decididos antes de que doliera, no después de un incidente. Y lo probamos con caos real, no solo con diseño: matamos el TaskManager a mitad de proceso y confirmamos que el checkpoint restauró el estado exacto, sin duplicar ni perder nada.*

> *Lo que reforzaría si esto fuera a producción: nunca tocamos event-time/watermarks de verdad — se evitó a propósito porque el problema no lo necesitaba, pero es un hueco real —, no hay observabilidad más allá de logs y checks manuales, y nunca se probó bajo carga real. Prefiero decir eso claramente a fingir que está todo cubierto.*

Si insiste y pide profundidad técnica

Tres anécdotas concretas que aguantan cualquier pregunta de seguimiento — están verificadas en vivo, no son teoría:

1. **El ADR que se equivocó y se corrigió por escrito** (Kinesis no soportado en PyFlink 2.x, ADR-0001 → ADR-0008) — demuestra que documentas premisas para poder detectar cuándo fallan, no para tener razón siempre.
2. **El dual-write evitado con CDC-derivado** (ADR-0006, patrón repetido de la Fase 2 a la Fase 5) — demuestra que reconoces el mismo problema de Kleppmann en dos capas distintas del mismo sistema.
3. **El fix documentado que no funcionó la segunda vez**, y hubo que escalar a resetear el volumen entero en vez de iterar la misma solución — demuestra que sabes distinguir "iterar la misma solución" de "la corrupción está un nivel más arriba".

Preguntas trampa a las que ya tienes respuesta honesta preparada

- *"¿Has trabajado con event-time y watermarks?"* → No en este proyecto, a propósito — explica por qué (§2.1) y qué construirías para cerrar ese hueco (un job con ventanas reales sobre datos out-of-order).
- *"¿Cómo monitorizáis esto en producción?"* → No hay observabilidad real hoy, es la brecha más grande identificada (§2.2) — y decirlo así es más fuerte que inventar un dashboard que no existe.
- *"¿Lo has probado bajo carga?"* → No, volumen de PoC únicamente — el hot-shard de ADR-0005 es un riesgo aceptado, no medido bajo tráfico real (§2.3).

---

4. Backpressure y reprocesamiento desde checkpoint/savepoint

Distingue lo verificado en este proyecto (recuperación desde checkpoint tras un fallo) de lo que solo se puede responder desde la teoría (backpressure sostenido bajo carga real) — nunca lo hemos tenido que gestionar de verdad porque el volumen de la PoC es pequeño y siempre se arranca en limpio. Es honesto decir eso en una entrevista, no inventar una anécdota que no ocurrió.

P: ¿Qué es backpressure en Flink y cómo se propaga?

R: Es la señal de que un operador downstream no puede consumir tan rápido como el upstream produce — el sink escribe más lento de lo que la ventana emite, o el `AsyncFunction` tiene su `capacity` de `unordered_wait` saturada de peticiones en vuelo. Flink no descarta datos ni los acumula sin límite en memoria: usa buffers de red con crédito (credit-based flow control) entre subtasks, así que cuando un operador se satura, deja de pedir crédito al de arriba, que a su vez deja de vaciar sus propios buffers, y la señal se propaga hacia atrás hasta la fuente (Kafka), que simplemente deja de avanzar el offset que consume. El pipeline se ralentiza de forma controlada en vez de caerse o perder datos.

P: ¿Cómo se detecta backpressure en producción?

R: Con métricas, no adivinando — el hueco que ya reconocemos en §2.2 (cero observabilidad real en este proyecto). Flink expone `backPressuredTimeMsPerSecond` por subtask en sus métricas, y la propia UI web marca visualmente qué operador está "OK/LOW/HIGH" en backpressure. La señal característica es que el operador *aguas abajo* del cuello de botella tiene baja utilización de CPU mientras el que está justo antes se

satura — el cuello de botella real está una posición más adelante de donde parece, porque el que se ve "trabajando al 100%" a menudo solo está esperando.

P: En nuestro propio Bloque 5 (Async I/O contra `price_decision`), ¿dónde aparecería backpressure si DynamoDB empezara a responder lento?

R: En `capacity` de `AsyncDataStream.unordered_wait` — ese número es exactamente el límite de cuántos lookups pueden estar pendientes a la vez por subtask. Si DynamoDB se ralentiza, las respuestas tardan más en volver, el buffer de `capacity` peticiones en vuelo se llena, y `async_invoke()` deja de poder aceptar bookings nuevos hasta que se libera un hueco — eso frena al operador de dedup justo antes, que frena al `KeyedProcessFunction`, que frena al consumo de `booking-events.v1`. Es el mismo mecanismo de crédito, aplicado a I/O externo en vez de a otro operador de Flink.

P: ¿Qué diferencia hay entre un checkpoint y un savepoint?

R: Un checkpoint es automático, periódico (`checkpoint_interval_ms` en este proyecto), pensado para recuperación de fallos — Flink lo gestiona él solo y los va rotando/limpiando. Un savepoint es manual, disparado explícitamente (`flink savepoint <job-id>`), pensado para operaciones planeadas: actualizar el código del job, cambiar el paralelismo, migrar de cluster. Ambos usan el mismo mecanismo interno (snapshot distribuido vía barreras de Chandy-Lamport) y el mismo formato de estado, pero un savepoint está pensado para durar y ser portable entre versiones del job; un checkpoint no tiene esa garantía de compatibilidad hacia adelante.

P: ¿Cómo se reprocesa desde un checkpoint, y qué garantiza que no se dupliquen ni se pierdan eventos?

R: Al reiniciar, Flink recarga el último checkpoint completado: el estado de cada operador (nuestros `ValueState` de dedup, los acumuladores de ventana a medio cerrar) y los offsets de Kafka que `KafkaSource` tenía confirmados en ese momento — no los offsets reales de commit en Kafka, los que Flink guardó dentro del propio checkpoint. Vuelve a leer desde ahí, así que **algunos eventos se reprocesan** (los que ya se habían consumido después del último checkpoint pero antes del fallo) — eso es exactamente por lo que el dedup por `ValueState[bool]` con TTL (Bloque 3/4 de esta fase) no es opcional: sin él, ese reprocesamiento produciría conteos duplicados. La garantía real que da Flink es "exactly-once en el estado interno" (el estado nunca queda a medias gracias al two-phase commit de las barreras), no "exactly-once en el mundo exterior" — eso último depende de que el sink sea idempotente o transaccional, que es justo lo que ya verificamos en la Fase 5 (AC-04, `docs/AUDIT_DIARY.md`): matar el consumer de Iceberg a mitad de stream, con un registro más escrito mientras estaba caído, y al reiniciar recuperó exactamente las filas esperadas sin duplicar ni perder ninguna.

P: Si el pipeline lleva horas caído (el productor no para de escribir mientras tanto), ¿qué pasa al recuperar?

R: Aquí es donde entra la retención, no solo el checkpoint. El checkpoint te dice *desde dónde* reanudar, pero si esos offsets ya cayeron fuera de la ventana de retención del topic (24h en Kafka, en este proyecto — ver §1, "doble bus de eventos"), esos mensajes ya no existen y no es un problema de Flink, es dato permanentemente perdido. Asumiendo que la retención cubre el hueco, Flink no "pone al día" instantáneamente: procesa el backlog acumulado tan rápido como el paralelismo se lo permita, y durante ese catch-up es *esperable* ver watermarks avanzando de golpe y ventanas cerrándose en ráfaga — no es un bug, es el pipeline recuperando terreno. Nunca hemos medido esto bajo carga real (mismo hueco que §2.3): sabemos explicarlo, no lo hemos cronometrado.